



Deep Residual Learning for Image Recognition

0. Abstract

- 기존보다 훨씬 더 깊은 신경망을 **안정적으로 학습**할 수 있도록 하는 **Residual Learning(잔차 학습) 프레임워크** 제안
 - 기존 : 각 레이어가 입력과 무관한 새로운 함수를 직접 학습하도록 설계됨
 - 논문 : 레이어들이 **입력에 대한 변화량(residual)** 을 학습하도록 구조를 재정의
- 네트워크는 어떤 함수 $H(x)$ 를 직접 학습하는 대신, 입력 x 를 기준으로 한 차이 $H(x)-x$ 를 학습하고, 최종 출력은 이를 다시 입력과 더하는 형태로 구성 → 깊은 네트워크의 최적화 훨씬 수월
- residual network가 **plain network**보다 **최적화가 쉽고**, 깊이를 크게 늘려도 성능 저하 없이 오히려 **정확도가 향상됨**
- ImageNet 데이터셋에서 최대 152-layer까지의 residual network를 학습, 기존 VGG 네트워크보다 약 8배 깊지만 계산 복잡도는 더 낮다. 이 모델들의 양상불은 ImageNet test set에서 3.57%의 error를 기록했고, ILSVRC 2015 classification 부문 1위
- CIFAR-10 데이터셋에서도 **100-layer** 및 **1000-layer** 수준의 매우 깊은 네트워크를 성공적으로 학습할 수 있음을 보였으며, 깊은 표현 자체가 성능 향상에 중요한 역할을 한다는 점을 실험적으로 확인
 - COCO 객체 검출 데이터셋에서도 28%의 상대적 성능 향상 달성
- Residual Network는 ImageNet 분류뿐 아니라 **ImageNet detection, localization**, 그리고 **COCO detection** 및 **segmentation** 과제에서도 모두 1위를 기록했다. 이러한 결과는 residual learning이 특정 문제나 데이터셋에 국한된 기법이 아니라, **다양한 시각 인식 문제 전반에 적용 가능한 일반적인 원리임을 보여줌**

1. Introduction

- CNN은 이미지 분류에서 큰 성과를 냈으며, 저수준-중간-고수준 특징을 end-to-end 구조로 학습
 - 네트워크의 **깊이(depth)**가 깊어질수록 표현력도 함께 증가
 - ImageNet 최고 성능 모델들은 대부분 **매우 깊은 네트워크** 구조를 사용
- 이에 따라 자연스럽게 제기되는 질문
 - **"더 좋은 성능은 단순히 레이어를 더 많이 쌓는 것으로 얻을 수 있는가?"**
- 과거에는 **vanishing / exploding gradient 문제**로 인해 깊은 네트워크 학습 자체가 어려웠음
 - 최근에는 적절한 초기화와 **Batch Normalization** 덕분에 수십 개 레이어도 SGD로 수렴 가능
- 그러나 학습이 가능해진 이후, 새로운 문제가 드러남 → **Degradation Problem**
 - 네트워크 깊이가 증가하면
 - 정확도가 포화되는 수준을 넘어서
 - 오히려 **성능이 급격히 저하**
 - 이는 overfitting 때문이 아니라
 - **훈련 오차(training error) 자체가 더 커지는 현상**
- 이 현상은 직관적으로 설명하기 어려움
 - 얇은 네트워크에 레이어를 추가하되 해당 레이어들이 **identity mapping**만 수행하도록 구성하면
 - 깊은 네트워크는 최소한 동일한 성능을 내야 함
 - 즉, 이론적으로는 깊은 모델의 해 공간이 얇은 모델을 포함
 - 그럼에도 실제 최적화(SGD + backprop)는 이런 "구성 가능한 해"를 효과적으로 찾지 못함
- 논문은 degradation 문제를 **표현력 부족 문제가 아니라 '최적화 문제'**로 해석
 - 깊은 plain network는 표현력은 충분하지만 최적화가 어려워 성능이 오히려 악화됨
- ⇒ 이를 해결하기 위해 **Residual Learning Framework** 제안
 - 레이어들이 새로운 함수를 처음부터 학습하도록 두는 대신
 - **입력에 대한 변화량(residual)**만을 학습하도록 구조를 변경

- 구체적으로
 - 기존 목표 함수: $H(x)$
 - Residual Learning: $F(x)=H(x)-x$ 를 학습 $\rightarrow$ 최종 출력은 $F(x)+x$
 - 만약 identity mapping이 최적에 가깝다면 여러 비선형 레이어로 identity를 억지로 구성하는 것보다 residual을 0에 가깝게 만드는 것이 최적화 측면에서 훨씬 쉬움
- 이 구조는 **shortcut connection**으로 구현
 - 입력을 몇 개 레이어를 건너뛰어 출력에 직접 더함
 - 추가 파라미터와 계산량이 거의 없음
 - 전체 네트워크는 기존과 동일하게 end-to-end 학습 가능
- 논문은 ImageNet, CIFAR-10 실험을 통해 다음을 보임
 - residual network는 plain network보다 **훨씬 깊어져도 안정적으로 학습**
 - 네트워크 깊이를 크게 늘렸을 때 실제로 **성능 향상을 얻을 수 있음**
- 특히 ImageNet에서는
 - 최대 **152-layer** residual network를 성공적으로 학습
 - 기존 VGG보다 훨씬 깊지만 계산 복잡도는 더 낮음
 - 분류뿐 아니라 객체 검출, 위치 추정, 분할 등 다양한 시각 인식 과제에서도 뛰어난 일반화 성능을 보임
- 결론적으로
 - residual learning은 특정 모델이나 데이터셋에 한정된 기법이 아니라
 - **깊은 신경망 최적화 전반에 적용 가능한 일반적인 원리임**을 제시

2. Related Work

2.1 Residual Representations

- 이미지 분석 분야

Vector Quantization : 원본 벡터 자체를 인코딩

$$x \rightarrow ck$$

VLAD : dictionary를 기준으로 잔차벡터를 인코딩

$$\text{residual} = x - ck$$

Fisher vector : VLAD의 확률적 버전

- Vector Quantization < VLAD / Fisher vector
- 원래 신호 자체보다 잔차벡터를 인코딩하는 것이 훨씬 효과적임

- 컴퓨터 그래픽스 분야

PDE(편미분 방정식)을 해결할때 Multigrid 방법 널리 사용됨

Multigrid : 문제를 여러 스케일로 나눔

: coarse scale 과 fine scale 사이의 잔차를 해결

Hierarchical basis pre-conditioning : 두 스케일간 잔차를 변수로 사

- 일반 방법 < Multigrid / Hierarchical basis pre-conditioning
- 문제를 잔차형태로 재구성하면 최적화가 훨씬 쉬워짐

2.2 Shortcut Connections

오래전부터 연구되어온 개

- MLP

입력 → 출력 직접 연결하는 선형레이어 추가

- gradient 안정화 연구

중간레이어 → 보조 분류 직접연결

- shortcut 기반 연구

layer response / gradient / error propagation 안정화

- inception

shortcut + deeper branch 구

- highway networks

gating function이 있는 shortcut (파라미터 0, 데이터 의존적)

gate가 닫히면 잔차가 아님

- Resnet

identity shortcut (파라미터 X)

항상 열려있음 → 정보 항상 전달

잔차함수 지속적으로 학습

3. Deep Residual Learning

3.1 Residual Learning Formulation

기존의 plain network에서는 여러 층으로 구성된 블록이 입력 x 에 대해 어떤 목표 함수 $H(x)$ 를 직접 학습하도록 설계되어 있다. 하지만 논문은 이 접근 자체가 깊은 네트워크에서 최적화를 어렵게 만든다고 본다.

Residual learning에서는 목표를 다음과 같이 재정의한다.

$$F(x) = H(x) - x \quad F(x) = H(x) - x$$

$$F(x) = H(x) - x$$

즉, 네트워크가 $H(x)$ 자체가 아니라 **입력 x 에 대한 변화량(residual)**인 $F(x)$ 를 학습하도록 한다. 최종 출력은 다음과 같이 계산된다.

$$y = F(x) + x = F(x) + x$$

$$y = F(x) + x$$

이 구조의 핵심 직관은 다음과 같다.

- 만약 최적의 함수가 identity mapping에 가깝다면
 $H(x) \approx x \quad H(x) \approx x$
- 기존 구조에서는 여러 비선형 레이어가 이를 **역지로 근사**해야 하지만
- residual learning에서는 $F(x) \approx 0 \quad F(x) \approx 0$ 이 되도록 학습하면 되므로
최적화가 훨씬 쉬워진다

논문은 degradation problem을 **표현력의 한계가 아니라 최적화의 어려움**으로 해석하며, residual learning은 바로 이 최적화 문제를 구조적으로 완화한다.

3.2 Shortcut Connections

Residual learning은 **shortcut connection (skip connection)**을 통해 구현된다.

입력 x 를 몇 개의 레이어를 건너뛰어 출력에 직접 더하는 방식이다.

$$y = F(x, \{W_i\}) + x = F(x, \{W_i\}) + x$$

$$y=F(x,\{W_i\})+x$$

여기서 $F(x,\{W_i\})$ 는 일반적으로 2~3개의 convolution layer로 구성된 residual block이며, x 는 어떠한 가중치도 거치지 않고 그대로 더해진다.

이 구조의 중요한 특징은 다음과 같다.

- 추가 파라미터가 거의 없음
- 계산량 증가가 미미함
- 전체 네트워크는 기존과 동일하게 **end-to-end backpropagation**으로 학습 가능
- shortcut은 정보와 gradient가 깊은 층까지 직접 전달되는 **경로(highway)** 역할을 수행

결과적으로 gradient가 여러 비선형 변환을 거치며 약해지는 현상을 완화하고, 매우 깊은 네트워크에서도 안정적인 학습을 가능하게 한다.

3.3 Identity vs Projection Shortcut

입력과 출력의 차원이 같은 경우, shortcut은 단순히 identity mapping으로 구현된다.

하지만 차원이 변하는 경우(예: feature map 수 증가, stride 적용)에는 차원 정합이 필요하다.

논문에서는 이를 위해 다음과 같은 방식을 제시한다.

- **Identity shortcut**
 - 차원이 같을 때 사용
 - 파라미터 추가 없음
- **Projection shortcut**
 - $1 \times 1 \times 1$ convolution을 사용해 차원 맞춤
 - stride 적용 가능
 - 파라미터는 증가하지만 표현력 향상

이 설계는 이후 실험 파트에서 성능과 효율성 측면에서 비교된다.

3.4 Residual Block 구조

논문은 두 가지 대표적인 residual block 구조를 제안한다.

(1) Basic Block

주로 얇은 네트워크(ResNet-18, 34)에 사용된다.

- 구조: $3 \times 3 \rightarrow 3 \times 3$
 $3 \times 3 \rightarrow 3 \times 3 \times 3 \rightarrow 3 \times 3$
- 각 convolution 뒤에 Batch Normalization과 ReLU 적용
- shortcut과 더한 뒤 다시 ReLU

(2) Bottleneck Block

아주 깊은 네트워크(ResNet-50, 101, 152)에 사용된다.

- 구조: $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$
 $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$
- 첫 번째 $1 \times 1 \times 11 \times 1$: 채널 수 감소 (차원 압축)
- 마지막 $1 \times 1 \times 11 \times 1$: 채널 수 복원
- 계산량을 크게 줄이면서도 깊이를 확장 가능

이 bottleneck 구조 덕분에 152-layer 네트워크가 VGG-19보다 훨씬 깊음에도 불구하고 연산량은 더 적은 모델로 구현될 수 있었다.

3.5 Residual Learning의 의미

Residual learning의 본질은 “새로운 표현을 매번 다시 학습”하는 것이 아니라,

이미 학습된 표현을 유지하면서 필요한 만큼만 수정하는 구조

에 있다.

이 관점에서 residual block은 기존 표현을 파괴적으로 바꾸는 모듈이 아니라,

미세 조정(refinement)을 수행하는 모듈로 해석할 수 있다.

논문은 이러한 구조가 깊은 네트워크 최적화를 가능하게 할 뿐 아니라,

실제로 깊이가 깊어질수록 성능이 향상되는 조건을 처음으로 실험적으로 입증했다는 점에서 중요한 의미를 가진다.

4. Experiments

ImageNet

1. 18개 레이어 & 34개 레이어의 Plain Net과 ResNet 비교

Plain Nets

ResNets

34개의 레이어로 했을 때 18 레이어보다 2.8% 차이로 34 레이어가 18 레이어보다 더 validation error, train error 모두 더 높았음. 좋은 성능을 발휘함. train error가 상당히 낮 BN을 이미 사용했기 때문에, gradient vanishing이 아닌 성능저하 문제임을 알 수 있음.

2. Shortcut Connection 옵션 비교

차원을 늘리면서 어떤 방식이 더 효과적인지 실험

- (A) 제로 패딩: 추가 파라미터 없이 0만 채워 차원 맞춤
- (B) 투영(Projection): 1*1 컨볼루션을 사용해 차원 맞춤 (파라미터 증가)
- (C) 모든 shortcut 투영: 차원이 같아도 모든 연결에 1*1 컨볼루션 적용

결과는 $C > B > A$ 순서로 성능이 미세하게 좋았음.

하지만 C와 B 차이가 크지 않은데, C는 파라미터가 너무 많아지기 때문에 효율성을 위해서 B를 사용하거나, 더 단순한 곳에는 A를 사용하는 것이 좋음

3. Bottleneck Architectures

- 기존 2개 레이어 대신 3개 레이어를 쌓는 방식
 - $3*3, 3*3 \rightarrow 1*1, 3*3, 1*1$ (1*1레이어가 차원을 한번 압축하면서 연산량을 줄임!)
- ResNet(50/101/152): 이 방식을 사용해 만든 152 레이어 모델이 VGG-19보다 연산량이 훨씬 적음.
- 깊이가 깊어질 수록 성능이 계속 향상. (저하 현상 X)

→ **결과**

- ResNet-152 단일 모델로 top-5 에러 4.49%
- 앙상블 모델은 3.57%를 기록하여 ILSVRC에서 1위

CIFAR-10

1. 심층 네트워크

- 일반 네트워크는 20 → 56 으로 가며 training error가 증가하며 성능이 나빠짐
- 반면 ResNet은 20, 32, 44, 56, 110까지 깊어지며 정확도 향상

- ResNet-110은 164만 개의 파라미터를 가지며 6.43% 에러율 기록. 학습 초기에는 수렴을 돕기 위해 0.01의 낮은 학습률로 워밍업 수행

2. ResNet-1202

- 최적화 어려움 없이 train error 0.1% 미만으로 수렴
- 하지만 test error는 110 layer 모델보다 약간 높은 7.93%. 이는 데이터셋 크기에 비해 모델이 너무 큰 과적합 현상으로 분석됨

3. Layer Response Analysis

- ResNet의 학습 성공 이유를 확인하기 위해 표준편차를 분석
- 일반 네트워크보다, 출력값의 크기가 전반적으로 작았음.
- 이는 잔차 함수는 0에 가까울 것이라는 초기 가설을 뒷받침함. 즉, 이 레이어들은 이미 학습된 정보를 크게 건드리지 않고 미세한 수정만 하는것

Object Detection

PASCAL VOC & MS COCO 데이터셋 (객체 탐지)

- Faster R-CNN이라는 탐지모델의 백본을 VGG-16에서 ResNet-101로 교체함
- 다른 변경없이 네트워크만 바꿨을 때, COCO에서 mAP@[.5, .95]가 6.0% 포인트 증가 (28% 향상)
- 이를 기반으로 ILSVRC & COCO 2015의 detection, localization, segmetation 모두 1위